

Taylor & Francis
Taylor & Francis Group

Journal of Computational and Graphical Statistics

ISSN: 1061-8600 (Print) 1537-2715 (Online) Journal homepage: https://www.tandfonline.com/loi/ucgs20

On Adaptive Regularization Methods in Boosting

Mark Culp, George Michailidis & Kjell Johnson

To cite this article: Mark Culp, George Michailidis & Kjell Johnson (2011) On Adaptive Regularization Methods in Boosting, Journal of Computational and Graphical Statistics, 20:4, 937-955, DOI: 10.1198/jcgs.2010.09076

To link to this article: https://doi.org/10.1198/jcgs.2010.09076

View supplementary material

Published online: 24 Jan 2012.

Submit your article to this journal

Article views: 174

View related articles

Citing articles: 1 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=ucgs20

Supplementary materials for this article are available online.
Please click the JCGS link at http://pubs.amstat.org.

On Adaptive Regularization Methods in Boosting

Mark CULP, George MICHAILIDIS, and Kjell JOHNSON

Boosting algorithms build models on dictionaries of learners constructed from the data, where a coefficient in this model relates to the contribution of a particular learner relative to the other learners in the dictionary. Regularization for these models is currently implemented by iteratively applying a simple local tolerance parameter, which scales each coefficient toward zero. Stochastic enhancements, such as bootstrapping, incorporate a random mechanism in the construction of the ensemble to improve robustness, reduce computation time, and improve accuracy. In this article, we propose a novel local estimation scheme for direct data-driven estimation of regularization parameters in boosting algorithms with stochastic enhancements based on a penalized loss optimization framework. In addition, k-fold cross-validated estimates of this penalty are obtained during its construction. This leads to a computationally fast and effective way of estimating this parameter for boosting algorithms with stochastic enhancements. The procedure is illustrated on both real and synthetic data. The R code used in this manuscript is available as supplemental material.

Key Words: Dictionary learning; Gradient boosting; Learning rate.

1. INTRODUCTION

Classification algorithms have had a long history in statistics and machine learning. Over the years, various parametric and nonparametric approaches have been proposed, for example, linear discriminant analysis, nearest-neighbor classifiers, perceptrons, and classification trees (Ripley 1996). These techniques rely on fitting a *single* model to the data, which establishes a classification rule for predicting new cases. In the early 1990s, boosting (Freund and Schapire 1996) and bagging (Breiman 1996) algorithms appeared in the literature, motivated by PAC learning theory (Valiant 1984), in which a number of weak classifiers (learners) are averaged to produce an *ensemble* classifier (learner). Specifically, boosting procedures generate base learners iteratively, using observation (case) weights

Mark Culp is Assistant Professor, Department of Statistics, West Virginia University, Morgantown, WV 26506 (E-mail: *mculp@stat.wvu.edu*). George Michailidis is Professor, Department of Statistics, University of Michigan, Ann Arbor, MI 48109. Kjell Johnson is Director, PharmaTherapeutics Statistics, Pfizer, Groton, CT 06340.

© 2011 *American Statistical Association, Institute of Mathematical Statistics, and Interface Foundation of North America*
Journal of Computational and Graphical Statistics, Volume 20, Number 4, Pages 937–955
DOI: 10.1198/jcgs.2010.09076

at each iteration to target misclassified observations from the previous iteration's base learner.[1] In addition, a coefficient is optimally obtained to scale a new base learner when added to the existing ensemble of learners. Bagging is another ensemble method in which bootstrapping (sampling of cases with replacement) is used to obtain many different base learners that are then combined through averaging (Breiman 1996). This procedure, in its original form, does not perform as strongly as boosting; however, successful hybrid algorithms have been developed to incorporate both boosting and bagging methodologies with improved performance (Friedman 2002; Rosset 2005; Culp, Michailidis, and Johnson 2010).

Recent advances in boosting involve local strategies to provide additional robustness and flexibility with the ultimate goal of improving performance. For example, to determine which weak learner is picked at a specific iteration, boosting can incorporate a greedy gradient descent search over a set of similar learners (known as a dictionary, e.g., dictionary of classification trees) (Friedman 2001; Rosset, Zhu, and Hastie 2004; Rosset 2005). This search involves optimization of a loss function to determine both the learner and the corresponding coefficient. Another simple modification to improve performance applied locally, that is, at each iteration, is to directly scale the coefficient of each term in the ensemble by a tolerance input parameter (known as the *learning rate*) (Friedman 2001). A third local modification applied to boosting is restricting either the variables or the cases considered during construction of the base learner using a random mechanism (Breiman 2001; Friedman 2002). This modification leads directly to hybrid boosting and bagging algorithms often referred to as stochastic boosting, where the randomization provides a computational speed-up, an additional improvement in accuracy, and robustness to a noisy response vector (Friedman 2002). It should be noted that the approach in the works of Friedman (2002) and Breiman (2001) are primarily applicable to regression trees, while several other approaches can be used with arbitrary base learner dictionaries (Freund and Schapire 1996; Rosset, Zhu, and Hastie 2004; Rosset 2005).

Each individual local enhancement discussed above has a global impact on the resulting ensemble. Different loss functions affect the sequence of learners chosen and the underlying learners comprising the ensemble. Application of the learning rate provides a regularization effect which allows the procedure to select more base learners without overfitting. As a result, the performance can improve or deteriorate depending on how this parameter is specified. The random mechanism applied at each iteration allows the method to efficiently search strong base learner dictionaries (here strong refers to learners typically with better misclassification rates than those selected in classical boosting). This random mechanism is often associated with reducing the variance of the ensemble relative to that of an individual learner (Breiman 1996, 2001; Friedman 2002). The use of strong learners allows the algorithm to attain low bias (Breiman 2001).

In this article, we present a data-driven estimation scheme for estimating the regularization parameters commonly used in regularized stochastic boosting algorithms. As pointed

[1] A base learner is used generally throughout the article, which refers to a classifier applied to the data trained with the current weight distribution.

out above, setting this parameter in boosting is difficult and it can have a significant impact on the performance. Specifically, we propose a local constrained optimization problem involving a loss function and a penalty term. The regularizer on the penalty term is then estimated at each boosting iteration, which allows it to adapt to the progress of boosting and to be determined from the data directly. In this regard, we investigate regularization in boosting with exponential loss and propose two interesting penalties: one which is differentiable at zero, and one which is not. For the first penalty, the ensemble is constrained to use each base learner generated by the ensemble no matter how it performs, whereas the second penalty allows the boosting algorithm to reject base learners whose performance does not exceed a threshold. Interestingly, the threshold is adjusted at each iteration allowing the admittance of the current candidate learner in the ensemble to depend on the progress up to that point in the ensemble. In addition, we consider a ridge penalty with exponential loss, showing that it leads to approximating the classic learning rate in this more general framework. The regularization parameter is estimated locally at each iteration which dramatically reduces the computation time for estimating it, when compared to global methods. The results are illustrated on both real and simulated data with a variety of base learner dictionaries (e.g., trees, neural nets, support vector machines).

The remainder of the article is organized as follows. Section 2 introduces the algorithm. In Section 3, we discuss the regularization framework and introduce two penalties compatible with exponential loss. Further, we outline a procedure for quickly obtaining cross-validated estimates for the shrinkage parameter. In Section 4, an empirical investigation of the performance of the new penalties is undertaken with simulated and real data. Some concluding remarks are drawn in Section 5.

2. PROBLEM SETUP AND THE REGULARIZED BOOSTING ALGORITHM

The setup for the binary classification problem is as follows: p feature vectors x_i have been collected for n cases, along with a binary response $y_i \in \{-1, 1\}$. The objective is to construct a classifier $h : x \to y$, such that future cases with feature vector $x^\star$ can be predicted with a high degree of accuracy. In the present context, it is assumed that $\mathcal{H}$ is a prespecified set of base learners, such as decision trees, neural nets, support vector machines, etc. The ensemble of learners $\{h_k\}_{m=1}^M$ is estimated from the training data $\{y_i, x_i\}_{i=1}^n$ using a bootstrapped sample and set of case weights $\{w_i\}$.

The final ensemble of classifiers is given by

$$f_M(x) = \sum_{m=1}^{M} \beta_m h_m(x) = \sum_{h \in \mathcal{H}} \beta(h) h(x), \tag{2.1}$$

where $\beta(h) = \beta_m I\{h = h_m \text{ for some } m \leq M\} \in \mathbb{R}$ and $\{h_m\}_{m=1}^M$ is the fixed sequences of base learners selected from $\mathcal{H}$. An example of an existing technique that fits in this framework is bagging (Breiman 1996). The bagging procedure iteratively fits base learners to random subsamples of the data with identical case weights and then forms the ensemble by averaging the selected learners. Formally, let $\{h_m\}_{m=1}^M$ be the sequence of base learners

selected; then, the ensemble is $f_M(x) = \sum_{h\in\mathcal{H}} \beta(h)h(x)$ for any base learner dictionary $\mathcal{H}$ with $\beta_m = \frac{1}{M}$.

The steps for obtaining an ensemble using *Regularized Stochastic Boosting* (RSB) are presented next. First, assign observation weights w_i as $-\frac{\partial L(y,f(x))}{\partial f(x)}$ evaluated at $(y_i, f_{m-1}(x_i))$ and subsequently normalized by their ℓ_1 norm. A basis function, $h_m \in \mathcal{H}$, is greedily selected using the weights on a bootstrapped random subsample of the data. The next step is to select the optimal coefficient based on the line search given by

$$\beta_m(h_m) = \arg\min_{\beta} \sum_{i=1}^{n} L(y_i, f_{m-1}(x_i) + \beta h_m(x_i)). \tag{2.2}$$

The final step updates the ensemble of functions by adding the new term

$$f_m(x) = f_{m-1}(x) + \beta_m h_m(x). \tag{2.3}$$

Then, the process is repeated M times. The strategy is justified as iteratively selecting a new term, $\beta_m(h_m)h_m(x)$, in the direction of steepest descent to minimize the loss in adding this term to the existing ensemble, $f_{m-1}(x)$.

As an example of RSB, consider using squared error loss (details given in Algorithm 1), $L(y, f) = (y - f)^2$. The weights are given by $w_i = |\epsilon_{m_i}|/\|\epsilon_m\|_1$, with ϵ_m representing the residual vector from response y to f_{m-1}. The base learner is selected using these weights on a random sample. For the line search, the coefficient is determined as the minimizer of $L(y, f_m + \beta_m(h_m)h_m(x)) = \sum_{i=1}^{n}(\epsilon_{m_i} - \beta_m(h_m)h_m(x_i))^2$. In this case, the procedure is optimally determining the next term, $\beta_m h_m$, closest to the residual, ϵ_m. Although this result is intuitive, the evidence given in the recent literature overwhelmingly suggests that squared error loss is not appropriate for classification problems since a loss is also incurred for correctly classified cases (Hastie, Tibshirani, and Friedman 2001; Rosset, Zhu, and Hastie 2004).

A more appropriate loss function in classification problems is the exponential loss $L(y, f) = e^{-yf}$. For this loss function, the weight updates are given by $w_i = y_i e^{-y_i f_{m-1}(x_i)}$ and normalized to obtain $w_i = \frac{e^{-y_i f_{m-1}(x_i)}}{\sum_j e^{-y_j f_{m-1}(x_j)}}$. The function, h_m, is selected next using these weights on a bootstrapped sample. The line search result is given by

$$\beta_m(h_m) = \frac{1}{2}\log\left(\frac{W_+(h_m)}{W_-(h_m)}\right),$$

Algorithm 1 Regularized Stochastic Boosting (RSB)

1: Initialize $f_0(x) := 0$ and input $\nu \in [0, 1]$
2: **for** $m = 1$ to M **do**
3: Set $w_i = -\frac{\partial L(y,g)}{\partial g}|_{(y_i, f_{m-1}(x))}$ and set $w_i = \frac{|w_i|}{\sum_j |w_j|}$
4: Fit $y_{\pi_m} = h_m(x_{\pi_m})$ using w_i where $h_m \in \mathcal{H}$ and π_m is a bootstrapped set of indices of the data size sampled with replacement
5: Compute $\beta_m(h_m) = \arg\min_\beta \sum_{i=1}^{n} L(y_i, f_{m-1}(x_i) + \beta h_m(x_i))$ (line search step)
6: Update $f_m(x) = f_{m-1}(x) + \nu\beta_m(h_m)h_m(x)$
7: **end for**

where $W_{\pm}(h) = \sum_{i=1}^{N} w_i I\{\pm y_i h(x_i) \geq 0\}$. Then the term $\beta_m h_m(x)$ is added to the existing ensemble to perform the final step. The steps given above provide an implementation for the classical AdaBoost algorithm given by Freund and Schapire (1996) using random bootstrapping to select the underlying base learner (typically classification trees).

For RSB, regularization is implemented by replacing the update step (2.3) with

$$f_m(x) = f_{m-1}(x) + \nu\beta_m(h_m)h_m(x) \tag{2.4}$$

for some $\nu \in [0, 1]$. Selecting a small value of ν slows the learning rate of boosting, which requires a larger M (i.e., more iterations) to improve the model's accuracy (Friedman 2001, 2002). It is difficult to obtain the optimal ν in practice, with the consensus being that $\nu \leq 0.1$ typically performs well (Friedman 2001). However, as noted by Mease and Wyner (2008) and illustrated in Section 4, this may not always be the case and specifying the wrong value can lead to a deterioration in performance even more pronounced than unregularized boosting. The full algorithm for RSB is presented as Algorithm 1.

Remark 1: It is noted that the Stochastic Gradient Boosting (SGB) algorithm provides another approach similar to RSB; both use the learning rate for regularization purposes (Friedman 2002) and optimize along the steepest descent direction. The main differences are that SGB uses a residual as the response in regression (as opposed to a weighted classifier), the line search is taken over bootstrapped examples, and this technique is primarily applicable with regression trees. Further, it is rather unclear how to set the learning rate, and therefore the ideas presented in this work for RSB would be applicable to SGB. Other approaches that the adaptive learning rate proposed in this work is applicable for include soft margins (Rätsch, Onoda, and Müller 2001) and generic gradient boosting (Rosset, Zhu, and Hastie 2004).

3. LOCAL REGULARIZATION

In the previous section, we defined a regularized boosting algorithm which required specification of the regularization parameter ν. In this section, we introduce a local regularization framework for estimating ν directly from the data. The rationale behind the proposed approach is that one can view the learning rate ν as directly shrinking a line search coefficient toward zero; a similar effect can be accomplished through a regularization framework.

Formally, given a local penalty function $J(\beta)$, $\beta \in \mathbb{R}$ (e.g., $J(\beta) = \beta^2$), one can then apply the constrained line search to obtain (assuming the solution is unique for each λ):

$$\beta_\lambda(h) = \arg\min_{\beta} \sum_{i=1}^{n} L(y_i, f(x_i) + \beta h(x_i)) + \lambda J(\beta) = \nu_\lambda(h)\beta_0(h), \tag{3.1}$$

which implies that

$$\nu_\lambda(h) = \frac{\beta_\lambda(h)}{\beta_0(h)}, \tag{3.2}$$

where $f(x)$ denotes the ensemble constructed after $m-1$ iterations. The *adaptive learning rate* is identified in the above result by $\nu_\lambda(h)$, which is the ratio of the regularized line

search coefficient divided by the unregularized one. The goal is to use the adaptive learning rate in place of ν in Algorithm 1. This requires two key conditions on the coefficient, $\beta_\lambda(h)$. The necessary requirements are motivated by the fact that for (2.4) one can simply select a constant ν as any value in the set $(0, 1]$. The following leads to a similar result for $\nu_\lambda(h)$ with $h \in \mathcal{H}$. Define $\tau(\lambda) = \beta_\lambda(\cdot)$ and assume the following two conditions hold:

$$\text{Condition 1:} \quad \tau(\lambda) \text{ is a nonincreasing continuous function in } \lambda. \tag{3.3}$$

$$\text{Condition 2:} \quad \operatorname{sign}(\tau(\lambda)) = c \text{ for any } \lambda \text{ with}$$
$$c \in \{-1, 1\} \text{ fixed whenever } |\tau(\lambda)| > 0. \tag{3.4}$$

Clearly $\tau(0) = \beta_0$ and so from (3.3) we have that (3.1) shrinks a coefficient from the unregularized version to zero. The sign condition (3.4) forces $\nu_\lambda(h) \geq 0$. From this, one can specify a finite λ so that $\nu_\lambda(h)$ will be any value in the set $(0, 1]$ for each $h \in \mathcal{H}$ (the case $\nu_\lambda(h) = 0$ depends on the regularization). Notice that this modification allows the learning rate to change with the function h, and this is why it is referred to as the adaptive learning rate. Below, we show that the loss functions and penalties considered in this section all satisfy these criteria.

3.1 THE HYPERBOLIC LASSO FOR EXPONENTIAL LOSS

The first loss and penalty functions considered provide a simple regularization setting for exponential loss. To start, the exponential loss function ($L(y, h) = e^{-yh}$) can be decomposed as follows:

$$\sum_{i=1}^{n} w_i L(y_i, \beta h(x_i)) = W_+(h)e^{-\beta} + W_-(h)e^{\beta}, \quad \text{where}$$

$$W_\pm(h) = \sum_{i=1}^{n} w_i I\{\pm y_i h(x_i) \geq 0\}.$$

Given the shape of the exponential loss function the penalty is designed to evaluate the coefficient on the hyperbolic scale,[2]

$$J(\beta) = e^{\beta} + e^{-\beta} - 2 = 2(\cosh(\beta) - 1). \tag{3.5}$$

The constant correction of -2 forces the function to pass through the origin. This penalty is referred to as the hyperbolic lasso (hlasso) since $\cosh(\beta) = \cosh(|\beta|)$ and therefore it is the lasso penalty transformed to the hyperbolic scale. Unlike the original lasso, this penalty is differentiable at zero. Solving (3.1) for β using exponential loss and the new penalty yields the following optimization problem:

$$\sum_{i=1}^{n} w_i L(y_i, \beta h_i) + \lambda J(\beta) = (W_-(h) + \lambda)e^{\beta} + (W_+(h) + \lambda)e^{-\beta} - 2\lambda.$$

[2]This particular penalty has also been used for penalizing exponential loss in the regression setting (Dekel, Shalev-Shwartz, and Singer 2005) and in semi-supervised graph-based learning (Culp and Michailidis 2008).

From this, the solution $\beta_\lambda(h)$ is given by simply taking the derivative and setting it to zero, which results in

$$\beta_\lambda(h) = \frac{1}{2}\log\left(\frac{W_+(h)+\lambda}{W_-(h)+\lambda}\right) \quad \text{for any } h \in \mathcal{H}. \tag{3.6}$$

It is easy to verify (3.3) and (3.4) are satisfied for (3.6) with any $h \in \mathcal{H}$.

The hyperbolic lasso (hlasso) (3.5) version of the adaptive learning rate results from use of this coefficient by the decomposition given in (3.2). The form of $\beta_\lambda(h)$ has some interesting interpretations. For example, if the weighted training error for a particular learner is near 0.5, then λ pulls the coefficient even closer to zero. However, if the error is particularly low, then the coefficient provides less regularization. Notice that the shrinkage nature of the hlasso is smooth, where for $W_-(h) < 0.5$ we have $\beta_\lambda(h) \to 0$ but will not equal 0.

3.2 THE EXPONENTIAL LASSO

The previous penalty was differentiable at zero and changes smoothly as a function of λ. Next we apply the ℓ_1 constraint on the exponential scale directly to provide another useful penalty for the proposed regularization framework. This penalty, unlike the hlasso, is not differentiable at zero and as a result it estimates coefficients directly at zero. Next, we investigate some of the properties of constraining the line search with the exponential lasso (elasso), $J(\beta) = e^{|\beta|} - 1$. Specifically, we solve

$$\min_\beta W_-(h)e^{\beta} + W_+(h)e^{-\beta} + \lambda\left(e^{|\beta|} - 1\right)$$

with $h \in \mathcal{H}$. In the following derivation we show that this loss and penalty function satisfy the conditions (3.3)–(3.4). The solution to the above optimization problem for a nonzero coefficient must satisfy the following:

$$W_-(h)e^{\beta} - W_+(h)e^{-\beta} + \lambda\,\mathrm{sign}(\beta)e^{|\beta|} = 0 \quad \text{where } h \in \mathcal{H}.$$

In the case that $\beta_\lambda(h) < 0$ we then have that

$$\begin{aligned}\beta_\lambda(h) &= \frac{1}{2}\log\left(\frac{W_+(h)+\lambda}{W_-(h)}\right) \times \mathbf{1}_{\{\beta_\lambda(h)<0\}} \\ &= \frac{1}{2}\log\left(\frac{W_+(h)+\lambda}{W_-(h)}\right) \times \mathbf{1}_{\{\lambda<2W_-(h)-1\}}.\end{aligned}$$

Similarly for $\beta_\lambda(h) \geq 0$ we have that

$$\beta_\lambda(h) = \frac{1}{2}\log\left(\frac{W_+(h)}{W_-(h)+\lambda}\right) \times \mathbf{1}_{\{1-2W_-(h)\geq\lambda\}}. \tag{3.7}$$

If $W_-(h) \leq 0.5$, then both $\beta_\lambda(h) \geq 0$ and $\beta_0(h) \geq 0$ ($\lambda = 0$) and similarly if $W_-(h) > 0.5$, then both $\beta_\lambda(h) \leq 0$ and $\beta_0(h) \leq 0$. From this and the closed form solution, we see that (3.4) is satisfied for all $h \in \mathcal{H}$. In either case, the indicator forces a continuous function not to change sign and hence (3.3) is satisfied.

In the case of exponential loss the learner selection step (Algorithm 1, step 4) forces $W_-(h) \leq 0.5$ which implies that (3.7) is always the solution for the elasso optimization

problem in Algorithm 1. The indicator is interpreted as enforcing a strong learner criterion where a learner's strength is given by $W_-(h) \leq \frac{1-\lambda}{2}$. In the case when this criterion is not met, the learner is then rejected from the weighting process and the corresponding coefficient is set to zero.

3.3 DIRECT SHRINKAGE AND ITS CONNECTION TO THE RIDGE PENALTY

The hlasso and elasso penalties each resulted in generalizing the original learning rate parameter ν using penalized optimization at the line search step. One natural interest is to determine whether employing the original learning rate ν directly in RSB can be equivalently obtained using regularization at the line search. In the following discussion, we establish that a ridge penalty for squared error loss optimization in regularized stochastic boosting (in the classification setting) does indeed obtain the learning rate ν exactly. In the case of exponential loss, we establish that regularization similar to the original learning rate is approximately obtained using the ridge penalty.

Consider the constraint optimization problem involving a squared error loss function and a ridge penalty term. Then, we have that

$$\begin{aligned} \beta_\lambda(h) &= \arg\min_\beta \sum_{i=1}^n (y_i - f(x_i) - \beta h(x_i))^2 + \lambda\beta^2 \\ &= \frac{\sum_{i=1}^n (y_i - f(x_i))h(x_i)}{\sum_{i=1}^n h_i^2 + \lambda} \qquad \text{and} \\ n &= \sum_{i=1}^n h(x_i)^2 = \nu_\lambda \beta_0(h) \quad \text{where} \\ \nu_\lambda &= \left(1 + \frac{\lambda}{n}\right)^{-1} \qquad \text{and} \qquad h \in \mathcal{H}. \end{aligned} \tag{3.8}$$

Therefore, selecting $\nu \in (0, 1]$, as previously determined, is equivalent to selecting $\lambda \in [0, \infty)$ using the ridge penalty for the line search step. In other words, regularization with the original learning rate as defined in (2.4) can still be obtained in this more general context.

Now, we consider the exponential loss function subject to a ridge penalty. The optimization problem takes the form

$$\min_\beta \sum_{i=1}^n e^{-y_i(f(x_i)+\beta h(x_i))} + \lambda\beta^2,$$

that gives

$$\begin{aligned} \beta_\lambda(h) &= \arg\min_\beta \sum_{i=1}^n w_i e^{-\beta y_i h(x_i)} + \lambda\beta^2 \\ &= \arg\min_\beta e^\beta W_-(h) + (1 - W_-(h))e^{-\beta} + \lambda\beta^2 \\ \Longrightarrow \qquad g_\lambda(\beta) &= e^{2\beta} W_-(h) - (1 - W_-(h)) + 2\lambda\beta e^\beta = 0. \end{aligned}$$

After some algebra involving a Taylor series expansion about $\beta_0 = \frac{1}{2}\log(\frac{1-W_-(h)}{W_-(h)})$ and solving for β_λ we get that

$$\beta_\lambda \approx \beta_0\left(\frac{(1-W_-(h))+\lambda\beta_0 e^{\beta_0}}{(1-W_-(h))+\lambda(\beta_0+1)e^{\beta_0}}\right) = \beta_0\left(1-\frac{\lambda}{C(W_-(h))+\lambda}\frac{1}{1+\beta_0}\right),$$

where it is easy to see that $C(W_-(h)) > 0$ assuming $W_-(h) < \frac{1}{2}$, and $C(W_-(h))$ is independent of λ. For small λ, the right-side approximation is close to β_λ and the ridge regularization is similar to multiplying the coefficient directly by the learning rate. The right-side approximation is less accurate for larger values of λ. However, the optimization functional is a convex regularization problem, and hence as $\lambda \to \infty$, $\beta_\lambda \to 0$. From this, larger values of λ result in significant shrinkage toward zero, which is similar to multiplying β_0 by a small constant, that is, the learning rate.

Finally, it is not hard to see that condition (3.3) is satisfied for this optimization problem. For the second condition, consider the following Newton update step to solve the penalized loss function iteratively: $\hat{\beta}_{k+1} = \hat{\beta}_k - \frac{g_\lambda(\hat{\beta}_k)}{g'_\lambda(\hat{\beta}_k)}$. Recall that $W_-(h) < \frac{1}{2}$ implies $\beta_0 > 0$. From this, and rearranging the Newton update, we observe that $\hat{\beta}_k$ maintains the same sign as $k \to \infty$. Therefore, the iterative solution to this problem will satisfy the sign condition in (3.4).

3.4 LOCAL CROSS-VALIDATION

We next discuss a procedure based on cross-validation for determining λ given a selected learner. The learning rate decomposition given in (3.2) results from the ratio of the regularized coefficient over the unregularized one. The k-fold estimate for the hlasso coefficient is given by

$$\begin{aligned}\beta_\lambda^{-\kappa(i)}(h) &= \frac{1}{2}\log\left(\frac{W_+^{-\kappa(i)}(h)+\lambda}{W_-^{-\kappa(i)}(h)+\lambda}\right), \quad \text{where}\\ W_\pm^{-\kappa(i)}(h) &= \frac{W_\pm(h) - \sum_{j\in\kappa(i)} w_j \mathbf{1}_{\{\pm y_j h(x_j)\geq 0\}}}{1-\sum_{j\in\kappa(i)} w_j},\end{aligned} \tag{3.9}$$

where $\kappa(i)$ are the observation indices over the folds. The elasso k-fold estimate is obtained in a similar manner. Using (3.9) one can quickly obtain coefficient updates. The cross-validated estimate in (3.9) is used to estimate λ by the following:

$$\lambda(h) = \arg\min_{\lambda\in[0,\infty)} \frac{1}{n}\sum_{i=1}^{\#fold}\sum_{j\in\kappa(i)} L\big(y_j, f_j + \beta_\lambda^{-\kappa(i)}(h)h(x_j)\big). \tag{3.10}$$

The advantage of applying local cross-validation techniques to estimate λ for ensemble algorithms is that it is simple and fast to compute.

4. EXPERIMENTAL INVESTIGATION

In this section, we investigate practical implementation aspects of the RSB algorithm. This includes an investigation of the effect of the two penalties, the merits of the proposed

regularization on data, assessing the effect of local cross-validation, and the comparison of using RSB with several common base learners (unless otherwise specified the base learners are classification trees).

4.1 EVALUATING THE EFFECT OF REGULARIZATION ON RSB

Two penalty terms were introduced for regularization purposes. Their main difference is that the elasso is not differentiable at zero, while the hlasso is. As a result, if λ is finite and the base learner's accuracy is greater than 0.5, then the elasso's adaptive learning rate can be zero, whereas the hlasso's learning rate cannot. In the ensuing analysis, we experimentally investigate the performance of RSB under these two penalties.

For the RSB algorithm with the adaptive learning rate, the amount of regularization is allowed to change from iteration to iteration. To quantify the overall amount of regularization, consider the following measure with $t \in [0, 1]$:

$$r_\lambda(t) = \frac{\sum_{m=1}^{M} I\{\nu_\lambda(h_m) \le t\}}{M}.$$

This measure records the relative frequency that the adaptive learning rate shrinks the coefficients less than t. For a fixed t, $r_\lambda(t)$ allows us to compare the rates of regularization for different penalties: the larger $r_\lambda(t)$, the more regularization up to t. In addition to $r_\lambda(t)$, consider $R_\lambda = \int_0^1 r_\lambda(t)\,dt = 1 - \frac{\sum_{m=1}^{M} \nu_\lambda(h_m)}{M}$, which provides the area under the regularization rate curve, or equivalently the average amount of regularization in the ensemble. When comparing penalties, the larger the value of R_λ, the greater the regularization. In practice, one would typically estimate the value of λ iteratively using (3.10); in this case, we denote by r_{cv} and R_{cv}, respectively, the pointwise and cumulative cross-validated regularization rates. To illustrate these measures, RSB was applied to a number of datasets from the UCI repository (Asuncion and Newman 2007); a chemical fingerprint and solubility dataset from pharmaceutical applications (Leach and Gillet 2003; Culp, Michailidis, and Johnson 2010); and a text dataset using the bag of words for the abstracts in computer science documents (McCallum et al. 1999). The results are given in Table 1 (averaged over 50 replications).

Table 1. $r_{cv}(0)$ and R_{cv} for the ensemble averaged over 50 values.

		hlasso		elasso	
Datasets	(n, p)	$r_{cv}(0)$	R_{cv}	$r_{cv}(0)$	R_{cv}
Thyroid	(215, 5)	0.000	0.646	0.471	0.664
Ion	(351, 34)	0.000	0.204	0.090	0.208
Fingerprint	(438, 152)	0.000	0.510	0.395	0.489
Cancer	(699, 9)	0.000	0.659	0.462	0.670
Pima	(768, 8)	0.000	0.293	0.069	0.284
Text	(879, 1543)	0.000	0.671	0.593	0.684
Credit	(1000, 34)	0.000	0.683	0.403	0.692
Solubility	(5631, 72)	0.000	0.453	0.263	0.459

The results in Table 1 come out as expected. The hlasso cannot estimate the adaptive learning rate directly at zero (refer to the $r_{cv}(0)$ column in Table 1), while the elasso can. As a result the elasso typically provides slightly more regularization over that of the hlasso for the entire ensemble (refer to the R_{cv} columns). However, the values are fairly close, which is not surprising since both techniques employ a similar mechanism for determining the sequence of learners and estimating λ (see remark below).

Next, we investigate the aggregate regularization rate with fixed λ using the UCI repository datasets. One practical advantage of the elasso over the hlasso is that the set of useful λ values is bounded between [0, 1] (i.e., $\lambda > 1 \Longrightarrow f_M(x) = 0$ for all x where f_M is the final ensemble). In the case of the hlasso, the potential values of λ are any finite positive real number, which makes computational selection of the range more difficult. We have found that for most datasets examined, λ values less than 40 tend to provide a fairly good range of choices for the hlasso penalty. The results for these data for both the hlasso and elasso versions of RSB for λ against R_λ are depicted in Figure 1. From this plot, we notice that the hlasso curve is fairly smooth in λ and quite similar for each dataset. For $\lambda > 5$ we observe that each value R_λ tends slowly toward the maximal value of 1 indicating nearly infinite regularization. In the case of the elasso, for $\lambda > 0$ we observe a jump in R_λ (note it must be zero for $\lambda = 0$ by definition) for each dataset. This jump gives each dataset a different starting point, which in turn affects the path of regularization as illustrated in the plot. Also, unlike the hlasso, the path does not have a particularly smooth shape, but is rather jagged at intermediate steps, for example, λ between 0.2 and 0.4 for the Thyroid dataset. This behavior of the elasso is directly related to the function not being differentiable at zero.

Remark 2: The implementation of local cross-validation is fairly easy to employ using grid search techniques due to the availability of the scalar estimate (3.10). However, the step size must be specified. Recall that for the elasso penalty, the parameter lies in [0, 1],

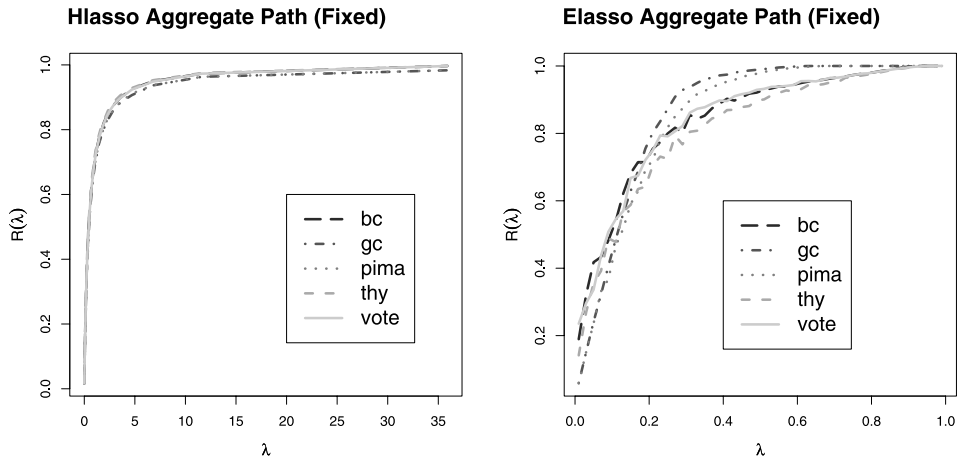


Figure 1. The average amount of regularization, R_λ, against λ for the five UCI datasets.

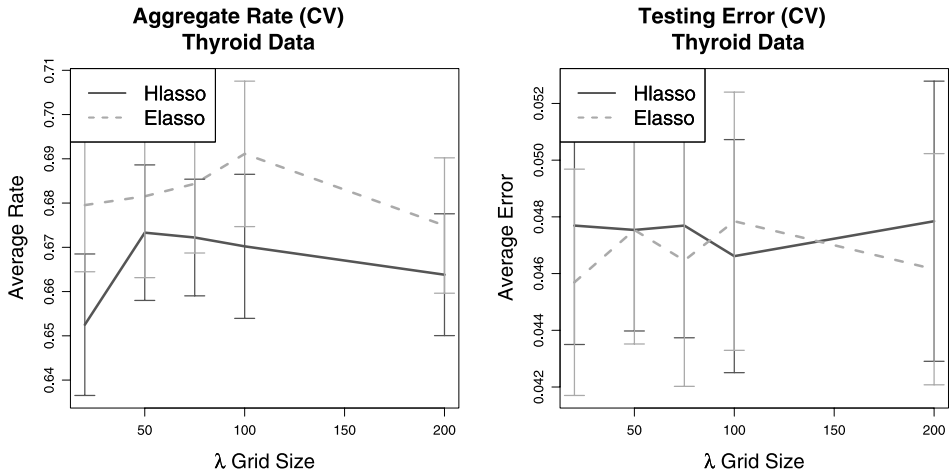


Figure 2. Cross-validated aggregate rate of regularization versus grid size (left). Testing error versus grid size (right).

while for the hlasso one, the range $[0, 40]$ proved quite effective in practice. The determination of the step size for the search is assessed using the Thyroid dataset. The number of grid partitions ranged from 25 to 200. The results shown in Figure 2 were based on 25 replications with 60% of the data used for training purposes and the remaining for testing. For the aggregate regularization rate (left plot) we observe slight fluctuations depending on the grid step size, where the elasso shows more (not significant) regularization than that of the hlasso. In the right plot, the testing error is shown, suggesting that it is not particularly influenced by the step size. This result indicates that a grid step ranging from 25 to 50 is sufficient for performing this regularization. Similar findings were recorded for other datasets (not shown here).

4.2 EFFECT OF THE ADAPTIVE LEARNING RATE

We assess this effect on both simulated and real data. The simulated data were generated by the following mechanism:

$$P(Y = 1|x) = q + (1 - 2q)\mathbf{1}_{\{\sum_{j=1}^{J} x^{(j)} > J/2\}}$$

with X uniformly distributed on the unit cube $[0, 1]^{20}$ of which $J = 5$ features are active and $q = 0.1$. Mease and Wyner (2008) proposed this scheme to illustrate that regularized boosting (with fixed $\nu = 0.1$) performed worse than unregularized boosting, which was a counterintuitive result. As we will show, allowing ν to be estimated locally provides more reasonable results.

In this simulation, we compare RSB using the hlasso and the elasso penalties, the original AdaBoost algorithm with $\nu = 0.1$ (shrinkage AdaBoost), and $\nu = 1.0$ (no shrinkage). The number of iterations was set to $M = 1000$ and the λ values were evenly spaced in the $[0, 40]$ grid with a grid size of 50 (refer to remark in Section 4.1). Clearly, Figure 3 (left) shows that shrinkage AdaBoost is competitive with the other methods within the first 200

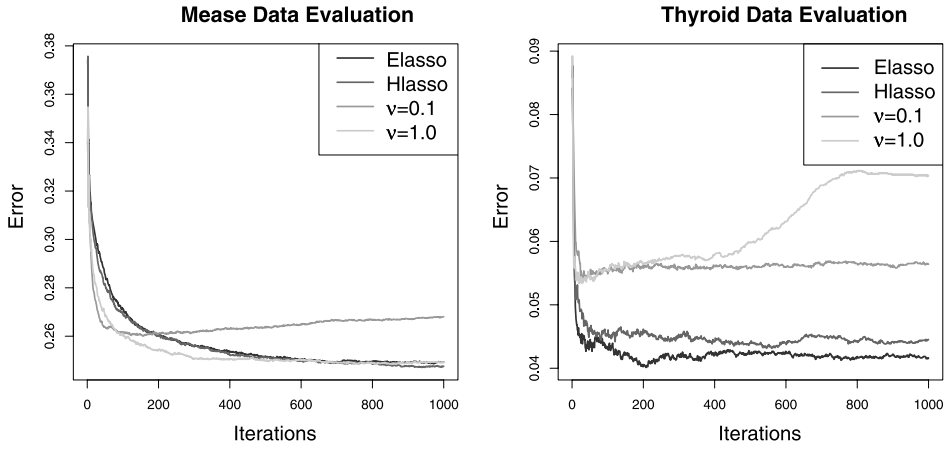


Figure 3. The accuracy results on the test data against iteration for the Mease data (left). The analogous results for the Thyroid data (right).

iterations. But its performance deteriorates rapidly after 200 iterations. Hlasso and elasso perform similarly and exhibit a slightly better performance than AdaBoost at more iterations. Thus, in this example, local shrinkage is not misled like global shrinkage. In addition to performance with respect to the test error, the measures $R_{cv}(\text{elasso}) = 0.280 \pm 0.002$ and $R_{cv}(\text{hlasso}) = 0.276 \pm 0.003$ indicate that the amount of regularization is similar between the two penalties. These results are consistent with Mease and Wyner's findings, and illustrate that allowing flexibility in the estimation of ν can improve the technique considerably.

As a follow-up comparison, consider a similar analysis for the Thyroid dataset (Figure 3(right)). For this comparison, the same algorithms were used under analogous conditions as in the previous example. Local regularization performs significantly better than no regularization or global regularization. For this dataset, the measures $R_{cv}(\text{elasso}) = 0.551 \pm 0.026$ and $R_{cv}(\text{hlasso}) = 0.521 \pm 0.020$ once again indicate that the amount of regularization is similar between the two penalties. Notice also that the values are nearly twice that of the Mease and Wyner data above.

The examples above illustrate that the RSB algorithm represents an advancement over boosting with fixed ν, since it does not need to be prespecified; instead the data can determine the optimal value of ν at each iteration.

4.3 EFFECT OF LOCAL CROSS-VALIDATION

We proposed local cross-validation to estimate the parameter λ at each iteration that requires fitting a single ensemble. Another approach to estimate the tuning parameter λ is to perform cross-validation directly on the ensemble. In other words, for 10-fold global cross-validation one could fit ten ensembles with λ varying over a grid of λ values, and then choose the one that minimizes the cross-validated error rate. The computational burden for this problem would require fitting $10 \times$ (the length of the grid for λ) ensembles relative to the one ensemble fit using local cross-validation. Due to this computational burden global cross-validation is rarely used in practice and hence this parameter is almost always spec-

ified directly by the user as noted above. Nevertheless, it is worthwhile to compare these two approaches.

The main issue under examination is whether the local cross-validation is expected to perform as well as global cross-validation on data. To assess this we employed regularized stochastic boosting using the elasso on the thyroid and the hlasso on the fingerprint data. For global cross-validation, each individual ensemble used for estimation of λ has a parameter M, which was in turn estimated using the out-of-bag error rate from the first 250 iterations with a burn in of 25 iterations. The results were averaged over 15 randomly selected testing and training sets with the training size set to 60% of the data.

In Figure 4 (left panel), we provide the average test error for the Thyroid dataset using both local and global cross-validation with the elasso. It can be seen that local cross-validation with the elasso performed better at the end of the ensemble while lagged in the beginning, although neither difference was statistically significant. A similar result for the hlasso is provided for the fingerprint data. This result establishes that the local cross-validation performs as strongly as the more computationally intensive global cross-validation approach. However, the average run for the thyroid data took 17 minutes for global cross-validation while for local cross-validation the average run took about 31 seconds using the same underlying RSB implementation in both cases. For the fingerprint dataset, which is less than twice the number of observations than in the thyroid data, the average run time for global cross-validation was 80 minutes relative to 1.3 minutes for local cross-validation. The time savings for local cross-validation provides an order of magnitude improvement over global cross-validation.

For the next comparison, we consider the problem of employing local cross-validation to fit ν directly at each iteration. It is important to note that currently a user will specify this parameter directly in boosting and that local cross-validation is not used to fit this parameter. In addition, recall from Section 3.3, this approach is equivalent to using a ridge penalty under squared error loss and approximately to using a ridge penalty under exponential loss, which provides new insights on interpreting the learning rate. Moreover, the hlasso

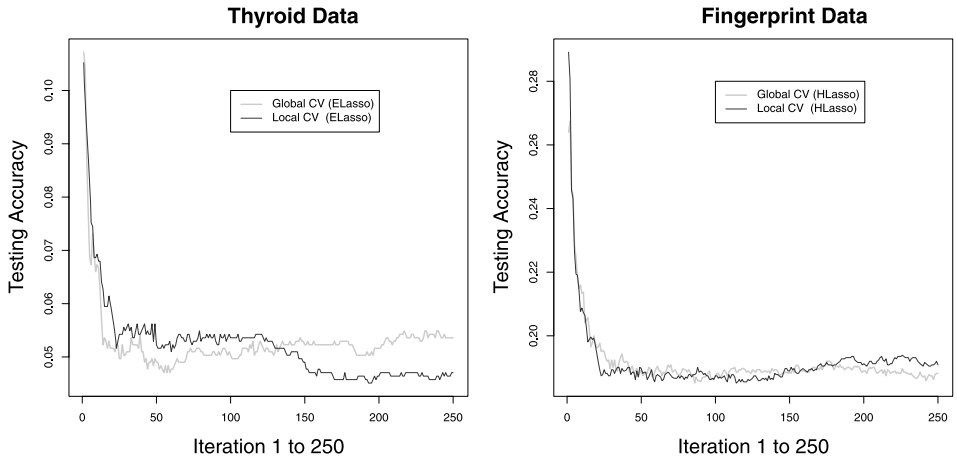


Figure 4. Global versus local cross-validation for the Thyroid Data (left) and Fingerprint (right).

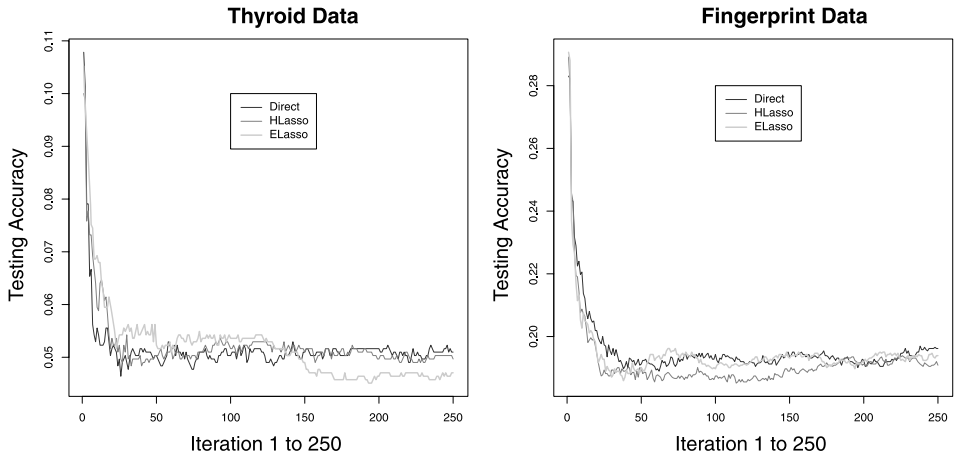


Figure 5. Performance evaluation of the direct estimation of ν with local cross-validation for the Thyroid Data (left) and Fingerprint (right).

and elasso penalties are more natural in scale with the exponential loss function, as opposed to a ridge penalty. Nevertheless, it is worthwhile to estimate ν directly and compare it to the hlasso and elasso penalties. The results are shown in Figure 5 for the Thyroid and Fingerprint datasets. From these results, we observe that estimation of ν directly with local cross-validation works reasonably well. This result is consistent in our experience in comparing different penalties at the line search (i.e., approximate ridge, hlasso, and elasso). The rationale is that each approach is using local cross-validation to estimate the internal parameter at each iteration and hence the technique is able to adapt to whatever penalty is used.

From these results, we observe that local cross-validation provides a computationally efficient approach. Existing approaches require specification of this parameter which can be difficult to justify in practice. The other option of global cross-validation is too expensive for real data.

4.4 EFFECT OF THE BASE LEARNER

For this comparison, we investigate the effect of different base learners on the performance of the RSB algorithm. The testing accuracy was reported for each dataset where 60% was used in training and the accuracy is the average over 25 iterations with M estimated out-of-bag out of 250 iterations with a burn in of 25 iterations. There are two comparisons that can be made: (i) comparison of RSB relative to the base learner, and (ii) comparison across different learners. For the first comparison, it will be established which base learners are expected to improve and which ones are not typically going to improve when using RSB. For the second comparison, we observe that any one technique can edge out any other depending on the dataset. However, all the approaches are usually competitive.

The base learners employed correspond to the standard R implementations (R Development Core Team 2006) of regression trees (Rpart) (Therneau, Atkinson, and Ripley 2007), neural nets (Nnet) (Venables and Ripley 2002), and support vector machines (SVM)

Table 2. Performance (testing error averaged over 25 replications) for varying tree depths.

Depth	hlasso	elasso
Stumps	0.256 (0.009)	0.263 (0.006)
Depth = 4	0.189 (0.007)	0.174 (0.005)
Depth = 8	0.173 (0.006)	0.167 (0.005)
Depth = 12	0.145 (0.003)	0.146 (0.003)
Pruning	0.112 (0.003)	0.117 (0.004)

(Dimitriadou et al. 2006). The tuning parameters are important, and each base learner requires different considerations along these lines. For the tree, we are primarily considering tree *depth* as the key parameter. It is noted that other tuning parameters for trees (e.g., *minsplit* and *cp*) could impact these results but the trade-off is the loss of simplicity and the fast execution times. In Table 2, we consider a variety of tree depths and also the impact of allowing internal pruning by the *rpart* routine for this parameter (Therneau, Atkinson, and Ripley 2007). The data for this example consisted of ten random normal variables $\{x_i\}$, where the response is given as

$$y_i = \begin{cases} 1, & \sum_{i=1}^{10} x_i^2 \geq 9.34 \\ -1, & \text{otherwise,} \end{cases}$$

where 9.34 is the median on the χ^2_{10} distribution (for more about these data refer to Hastie, Tibshirani, and Friedman 2001, chapter 10).

This result justifies using the internal rpart functionality to determine tree-depth and is used in the accuracy table. The neural net has several parameters with the most important being its *size*. We used 10-fold cross-validation with *size* in the range (2, 5, 10, 20, 30, 50). For some of the datasets, size had to be restricted as needed to a smaller range to accommodate binary variables. For the text data, the p larger than n case posed a problem where the neural net did not converge. The SVM required specification of two key parameters, the cost C and the kernel parameter γ. This technique was the most sensitive to tuning parameter settings and required an extensive two-dimensional search using 10-fold cross-validation. The parameters were estimated with $(C, \gamma) \in (0.1, 0.2, \ldots, 0.9, 1, 2, \ldots, 7) \times (0.1, \ldots, 0.9, 1)$. This was significantly more time consuming than the other techniques but allowed us to truly assess the performance of this approach. For the solubility dataset the search was computationally impractical. The RSB algorithm used the base learner settings throughout for the Nnet and SVM and hence the parameter search was only performed once per iteration. The resulting averages with standard errors are given in Table 3. The differences between hlasso and elasso are nearly negligible, which is intuitive since cross-validation is used for estimation in both cases. From these results, it is noted that tree-based learners provide potentially large improvements when used in RSB, neural nets provide some improvement, and SVMs typically provide no improvement and can make things worse. In the cross-comparison study, the results come out as expected where on some datasets SVMs edge out the other approaches while with others the RSB with trees or NNets performs the best. Computationally, the

Table 3. Performance comparison of regularization schemes with average and standard errors.

		Tree			Nnet			SVM			Tree
		Tree	hlasso	elasso	Nnet	hlasso	elasso	SVM	hlasso	elasso	AdaBoost
Thy	Ave	0.098	0.048	0.046	0.056	0.046	0.044	0.050	0.052	0.047	0.058
	SE	0.008	0.006	0.005	0.004	0.005	0.006	0.005	0.004	0.005	0.007
Ion	Ave	0.127	0.079	0.080	0.108	0.100	0.096	0.061	0.071	0.074	0.091
	SE	0.004	0.004	0.004	0.007	0.005	0.005	0.003	0.004	0.004	0.004
Finger	Ave	0.234	0.182	0.187	0.248	0.207	0.213	0.195	0.193	0.189	0.210
	SE	0.007	0.004	0.004	0.050	0.041	0.043	0.006	0.005	0.005	0.006
Cancer	Ave	0.059	0.038	0.039	0.058	0.048	0.050	0.038	0.057	0.057	0.046
	SE	0.003	0.002	0.002	0.003	0.003	0.003	0.003	0.002	0.002	0.002
Pima	Ave	0.260	0.254	0.252	0.346	0.320	0.330	0.241	0.266	0.269	0.247
	SE	0.004	0.003	0.003	0.005	0.005	0.006	0.004	0.006	0.006	0.003
Text	Ave	0.326	0.283	0.283	–	–	–	0.332	0.283	0.286	0.285
	SE	0.005	0.004	0.004	–	–	–	0.005	0.004	0.005	0.005
Credit	Ave	0.271	0.254	0.258	0.290	0.244	0.252	0.265	0.287	0.287	0.313
	SE	0.004	0.004	0.004	0.005	0.004	0.007	0.006	0.006	0.005	0.006
Sol	Ave	0.274	0.232	0.232	0.363	0.298	0.300	–	–	–	0.241
	SE	0.002	0.002	0.002	0.007	0.004	0.005	–	–	–	0.002

SVM approach took over twice as long in most cases to run due to the extensive parameter search necessary to achieve strong performance.

5. CONCLUSIONS

In this work, we introduced a Regularized Stochastic Boosting (RSB) algorithm, which employs a gradient descent search to select learners in a dictionary and fit an ensemble model using the learners. Regularization for this algorithm is employed by selecting a learning rate parameter, which can improve performance but requires user specification. One of the main objectives was to provide a penalized local estimation scheme for data-driven estimation of the parameters of interest. Relative to global cross-validation, local estimation is extremely computationally efficient, which makes estimation practically feasible. The next step involved determining combinations of loss functions and penalties well suited for this approach. Two appropriate penalty choices for the exponential loss function were introduced. The empirical investigation indicates that the adaptive learning rate provided a better performance over fixed choices of the parameter. Further, the algorithm is best suited for training with trees as the base learner but can work with arbitrary base learners. The study of other combinations of loss functions and penalty terms is a topic of future work.

SUPPLEMENTAL MATERIALS

RSB Routine: The R source code for performing Regularized Stochastic Boosting. (rsb.R)

Example: An example for using the RSB software. (ex_rsb.R)

ACKNOWLEDGMENTS

The authors thank the editor, an associate editor, and a reviewer for their insightful comments which led to a substantial improvement of the manuscript.

[Received May 2009. Revised June 2010.]

REFERENCES

Asuncion, A., and Newman, D. (2007), "UCI Machine Learning Repository," University of California, Irvine, School of Information and Computer Sciences, available at *http://www.ics.uci.edu/~mlearn/MLRepository.html*. [946]

Breiman, L. (1996), "Bagging Predictors," *Machine Learning*, 24 (2), 123–140. [937-939]

——— (2001), "Random Forests," *Machine Learning*, 45 (1), 5–32. [938]

Culp, M., and Michailidis, G. (2008), "Graph-Based Semisupervised Learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30 (1), 174–179. [942]

Culp, M., Michailidis, G., and Johnson, K. (2010), "The Ensemble Bridge Algorithm: A New Modeling Tool for Drug Discovery Problems," *Journal of Chemical Information and Modeling*, 50 (2), 309–316. [938,946]

Dekel, O., Shalev-Shwartz, S., and Singer, Y. (2005), "Smooth ϵ-Insensitive Regression by Loss Symmetrization," *Journal of Machine Learning Research*, 6, 711–741. [942]

Dimitriadou, E., Hornik, K., Leisch, F., Meyer, D., and Weingessel, A. (2006), "e1071: Misc Functions of the Department of Statistics (e1071), TU Wien," R package version 1.5-16. [952]

Freund, Y., and Schapire, R. (1996), "Experiments With a New Boosting Algorithm," in *International Conference on Machine Learning*, Boston, MA: Morgan Kaufmann, pp. 148–156. [937,938,941]

Friedman, J. (2001), "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics*, 29 (5), 1189–1232. [938,941]

——— (2002), "Stochastic Gradient Boosting," *Computational Statistics & Data Analysis*, 38 (4), 367–378. [938,941]

Hastie, T., Tibshirani, R., and Friedman, J. (2001), *The Elements of Statistical Learning: Data Mining, Inference and Prediction* (1st ed.), New York: Springer-Verlag. [940,952]

Leach, A., and Gillet, V. (eds.) (2003), *An Introduction to Chemoinformatics*, London: Kluwer Academic. [946]

McCallum, A., Nigam, K., Rennie, J., and Seymore, K. (1999), "Automating the Construction of Internet Portals With Machine Learning," *Information Retrieval Journal*, 3, 127–163. [946]

Mease, D., and Wyner, A. (2008), "Evidence Contrary to the Statistical View of Boosting," *Journal of Machine Learning Research*, 9, 131–156. [941,948]

R Development Core Team (2006), *R: A Language and Environment for Statistical Computing*, Vienna, Austria: R Foundation for Statistical Computing, aivailable at *http://www.R-project.org*. [951]

Rätsch, G., Onoda, T., and Müller, K. (2001), "Soft Margins for AdaBoost," *Machine Learning*, 42 (3), 287–320. [941]

Ripley, B. (1996), *Pattern Recognition and Neural Networks*, New York: Cambridge University Press. [937]

Rosset, S. (2005), "Robust Boosting and Its Relation to Bagging," in *KDD'05: Proceeding of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery in Data Mining*, Chicago, IL: ACM, pp. 249–255. [938]

Rosset, S., Zhu, J., and Hastie, T. (2004), "Boosting as a Regularized Path to a Maximum Margin Classifier," *Journal of Machine Learning Research*, 5, 941–973. [938,940,941]

Therneau, T., Atkinson, B., and Ripley, B. (2007), "rpart: Recursive Partitioning," R package version 3.1-36. [951,952]

Valiant, L. (1984), "A Theory of the Learnable," in *Proceedings of the 16th Annual ACM Symposium on Theory of Computing*, New York: ACM, pp. 436–445. [937]

Venables, W. N., and Ripley, B. D. (2002), *Modern Applied Statistics With S* (4th ed.), New York: Springer. [951]